广告 Agent Team PRD

1. 项目概述

1.1 项目名称

广告 Agent Team

1.2 项目定位

广告 Agent Team 是一套基于 OpenClaw 构建的广告项目多 Agent 协作系统。

它的目标不是做一个“会聊天的广告机器人”，而是构建一个面向广告项目生产流程的 AI 工作系统，将广告需求从输入、理解、研究、策略、审核、创意、媒介到最终交付，组织为一条可控、可复用、可校准、可持续优化的协作链路。

1.3 核心业务问题

广告项目天然依赖多岗位协同，但传统协作中长期存在以下问题：

- brief 信息不完整，前期沟通成本高
- 多岗位反复转述，信息损耗明显
- 洞察、策略、创意、媒介之间缺少稳定标准作业流
- 一处修改往往带来整单重做，返工成本高
- 历史项目难以沉淀为后续可复用资产

1.4 项目目标

构建一套稳定的广告 Agent Team，使广告需求能够沿固定工作流完成自动协作交付，并逐步具备以下能力：

- 固定主链路协作
- 单 specialist 灵活调用
- 节点级确认 Gate
- 多轮反馈与局部返工
- 文件型项目状态管理
- Reviewer 质量校验与人工校准
- 后续知识库、RAG 与数据复盘能力扩展

1.5 产品价值

该系统解决的不是单点内容生成问题，而是广告项目的协作组织问题。其核心价值包括：

- 降低项目启动与 brief 对齐成本

- 减少岗位间信息损耗
- 建立稳定的跨节点输入输出标准
- 在关键节点前置质量校验，降低后续返工
- 让项目过程、反馈和产物形成可复盘资产

---

2. 产品范围

2.1 系统边界

系统面向广告项目生产流程，当前聚焦以下核心能力：

- 需求输入与项目初始化
- 洞察研究
- 传播策略生成
- 策略审核与放行
- 创意执行输出
- 媒介与投放建议
- 最终交付汇总

2.2 Agent 范围

| Agent | 职责定位 |

|---|---|

| PM Agent | 唯一对外接口、工作流编排、Gate 管理、最终交付 |

| Insight Agent | 公开信息聚合、案例与趋势研究 |

| Strategy Agent | 传播命题、策略框架、信息结构 |

| Reviewer Agent | 只审 `strategy.md`，做质量判断与放行建议 |

| Creative Agent | 创意方向、文案与执行表达 |

| Media Agent | 渠道组合、投放节奏、达人与投流建议 |

2.3 工作流范围

系统支持两类工作模式：

- `standard_workflow`
- `single_specialist_workflow`

2.4 非目标范围

以下能力属于后续扩展，不纳入当前核心范围：

- Content Agent 平台内容适配全量接入
- Data Agent 自动复盘与优化闭环
- 各 Agent 独立知识库正式接入
- RAG 检索系统正式建设
- 自由编排式无限组合链路

---

3. 核心设计原则

3.1 模式决定路径

系统只保留两种工作模式，不做无限自由编排，优先保证链路稳定、可解释、可验证。

3.2 PM 只负责编排，不负责专业产出

PM 负责需求理解、模式判断、项目初始化、调度 specialist、管理 Gate、路由反馈和汇总交付，但不直接补做策略、创意和媒介内容。

3.3 角色边界必须清晰

每个 specialist 只在自己的职责边界内工作，避免节点之间相互污染：

- Insight 不代做 Strategy
- Strategy 不代做 Creative / Media
- Reviewer 不重写 Strategy
- Creative 不回写 Strategy
- Media 不补写 Strategy / Creative

3.4 确定性逻辑下沉到脚本

系统采用“prompt + script”混合编排：

- prompt 负责角色、边界、原则
- script 负责状态更新、任务构建、评分对比、文档同步等确定性逻辑

3.5 质量前置，而不是交付后补救

Strategy 不直接流向下游，而是必须先经过 Reviewer；只有通过质量校验的上游输入，才允许进入 Creative 和 Media。

3.6 人在回路中，而不是盲信 Agent

机器评审不是终点，人工复核与分差分析是质量体系的一部分。系统目标不是立刻替代专业人，而是建立一套可校准、可比较、可持续优化的判断机制。

---

4. 系统总体架构

4.1 总体结构

系统由 4 层组成：

1. 编排层

- `openclaw.json`
- `workspace/AGENTS.md`

负责：

- agent 注册
- 工作流规则
- Gate 规则
- 路由与调度原则

2. Specialist 层

- `workspace-insight/AGENTS.md`
- `workspace-strategy/AGENTS.md`
- `workspace-reviewer/AGENTS.md`
- `workspace-creative/AGENTS.md`
- `workspace-media/AGENTS.md`

负责：

- 角色边界
- 输入读取规则
- 输出结构
- 禁止事项

3. 工作流脚本层

- [init_project.py](/Users/ray/.openclaw/workspace-projects/init_project.py)
- [workflow_helper.py](/Users/ray/.openclaw/workspace-projects/workflow_helper.py)
- [task_builder.py](/Users/ray/.openclaw/workspace-projects/task_builder.py)
- [review_calibration.py](/Users/ray/.openclaw/workspace-projects/review_calibration.py)

- [wecom_doc_sync.py](/Users/ray/.openclaw/workspace-projects/wecom_doc_sync.py)
- [wecom_gate_render.py](/Users/ray/.openclaw/workspace-projects/wecom_gate_render.py)

负责：

- 新项目创建
- 状态流转
- specialist 任务包生成
- 人工评分记录与 machine vs human 对比
- 企业微信文档镜像与固定模板展示

4. 项目状态层

每个项目目录存放完整过程文件，包括：

- `brief.md`
- `state.json`
- `feedback.md`
- `insight.md`
- `strategy.md`
- `strategy_review.md`
- `strategy_review.json`
- `strategy_human_review.md`
- `strategy_human_review.json`
- `strategy_review_compare.md`
- `creative.md`
- `media.md`
- `delivery.md`

4.2 架构核心

该架构的核心不是“多个 Agent 并列存在”，而是：

- PM 负责统一编排
- specialist 负责按边界生产
- script 保证确定性执行
- 项目目录承接状态与产物
- Reviewer 负责策略质量门

---

5. 工作流设计

5.1 标准工作流

固定主链为：

```
PM -> Insight -> Strategy -> Reviewer -> Creative -> Media -> Delivery
```

5.2 标准工作流步骤

Step 1：用户输入

用户向 PM 提交广告 brief。

Step 2：项目初始化

PM 调用 [init_project.py](/Users/ray/.openclaw/workspace-projects/init_project.py)，创建唯一项目目录并初始化：

- `brief.md`
- `state.json`
- `feedback.md`

Step 3：Insight

PM 调起 Insight：

- 输入：`brief.md`
- 输出：`insight.md`

Insight 完成后进入 `insight_review` Gate。

Step 4：Strategy

PM 调起 Strategy：

- 输入：`brief.md + insight.md`
- 输出：`strategy.md`

Strategy 完成后直接进入 Reviewer，不单独放行下游。

Step 5：Reviewer

PM 调起 Reviewer：

- 输入：`brief.md + strategy.md (+ insight.md)`
- 输出：
  - `strategy_review.md`
  - `strategy_review.json`

Step 6：Reviewer Gate

Reviewer Gate 是标准主链中的质量门。此时用户可执行：

- `继续`
- `修改 strategy: ...`
- `人工评分: ...`
- `补充信息: ...`
- `终止`

Step 7：Creative

Reviewer Gate 通过后，PM 调起 Creative：

- 输入：`brief.md + strategy.md`
- 输出：`creative.md`

Creative 完成后进入 `creative_review` Gate。

Step 8：Media

Creative Gate 通过后，PM 调起 Media：

- 输入：`brief.md + strategy.md + creative.md`
- 输出：`media.md`

Media 完成后进入 `media_review` Gate。

Step 9：Delivery

Media Gate 通过后，PM 汇总生成 `delivery.md` 并向用户交付最终结果。

5.3 单节点工作流

当用户明确表示“只需要某一个 specialist 的输出，不要跑前序节点”时，PM 切换到 `single_specialist_workflow`。

当前支持：

- `insight-only`
- `strategy-only`
- `creative-only`
- `media-only`

其规则为：

1. PM 只调起目标 specialist
2. 不自动补跑前序链路
3. specialist 只读取当前项目允许范围内的 PM 上下文
4. 如信息不足，先返回最少必要补充问题

5. 用户补充后，PM 重跑同一 specialist
6. 输出完成后进入该节点对应 Gate

5.4 Gate 机制

Gate 是该系统的关键交互层，承担三类职责：

- 节点成果确认
- 多轮反馈入口
- 下游放行或局部返工控制

系统中的 Gate 不是简单停顿，而是工作流控制点。

---

6. Agent 角色定义

| Agent | 输入 | 输出 | 不负责 |

|---|---|---|---|

| PM | 用户需求、项目状态、各节点文件 | 路由、Gate、最终交付 | 直接产出专业内容 |

| Insight | `brief.md` | `insight.md` | 策略、创意、媒介判断 |

| Strategy | `brief.md + insight.md` | `strategy.md` | slogan、脚本、媒介投放 |

| Reviewer | `brief.md + strategy.md (+ insight.md)` | `strategy_review.md + strategy_review.json` | 重写策略、产出创意/媒介内容 |

| Creative | `brief.md + strategy.md` | `creative.md` | 重写策略、输出投流建议 |

| Media | `brief.md + strategy.md + creative.md` | `media.md` | 重写传播命题、产出创意文案 |

补充原则：

- Reviewer 只审核 Strategy
- Reviewer 不进入 `single_specialist_workflow`
- Creative 和 Media 只基于已通过的 Strategy 工作

---

7. 质量系统与 Reviewer 机制

7.1 Reviewer 的定位

Reviewer 不是附属 agent，而是 Strategy 到下游之间的质量门。

其职责是判断：

- 这份 Strategy 是否真正回应 brief
- 这份 Strategy 是否能被下游直接接住
- 这份 Strategy 是否具备放行条件

7.2 机器评审维度

Reviewer 使用 4 项维度评分，每项 1–5 分，满分 20 分：

| 维度 | 审什么 |

|---|---|

| Brief 匹配 | 是否真正回应用户需求，关键要求有无遗漏 |

| 策略质量 | 命题是否清晰、逻辑是否自洽、差异化是否成立 |

| 下游可用性 | Creative / Media 能否直接接手 |

| 交付完整性 | 是否存在占位符、缺字段、越界、明显漏洞 |

7.3 Hard Gate

命中任一项即 `FAIL`：

1. 占位符
2. 用户要求的关键字段缺失
3. 角色越界
4. 明显事实错误或无效链接

7.4 决策规则

| 条件 | 决策 |

|---|---|

| Hard Gate FAIL | REJECT |

| Hard Gate PASS + 总分 ≤ 11 | REJECT |

| Hard Gate PASS + 总分 12–15 | REVISE |

| Hard Gate PASS + 总分 ≥ 16 | PASS |

7.5 规则外置

Reviewer 的评审标准与输出格式已拆分为独立规则文件：

- [REVIEW_CRITERIA.md](/Users/ray/.openclaw/workspace-reviewer/REVIEW_CRITERIA.md)
- [REVIEW_OUTPUT_FMT.md](/Users/ray/.openclaw/workspace-reviewer/REVIEW_OUTPUT_FMT.md)

这样做的意义在于：

- 评分规则与 agent prompt 解耦
- 质量标准更容易独立迭代
- 评分机制可以逐步沉淀为长期质量资产

7.6 Human Calibration Loop

系统支持人工评分闭环：

1. 机器 Reviewer 输出：
   - `strategy_review.md`
   - `strategy_review.json`
2. 专业人工在 Reviewer Gate 输入同维度评分
3. 系统通过 [review_calibration.py](/Users/ray/.openclaw/workspace-projects/review_calibration.py) 生成：
   - `strategy_human_review.json`
   - `strategy_human_review.md`
   - `strategy_review_compare.md`
4. 用分差数据持续迭代 Reviewer prompt、评分说明和 hard gate

该闭环的目标不是让机器立刻替代专业人，而是建立“机器评审是否接近专业判断”的可校准体系。

---

8. 项目目录层与脚本化编排

8.1 设计原则

系统采用文件型项目状态管理，不依赖额外数据库。

8.2 项目目录结构

```
/Users/ray/.openclaw/workspace-projects/
├── _active_project.json
├── _templates/
│   ├── brief.template.md
│   └── state.template.json
├── init_project.py
├── workflow_helper.py
├── task_builder.py
├── review_calibration.py
├── wecom_doc_sync.py
├── wecom_gate_render.py
```

```
├── project-YYYYMMDD-001-name/
│   ├── brief.md
│   ├── state.json
│   ├── feedback.md
│   ├── insight.md
│   ├── strategy.md
│   ├── strategy_review.md
│   ├── strategy_review.json
│   ├── strategy_human_review.md
│   ├── strategy_human_review.json
│   ├── strategy_review_compare.md
│   ├── creative.md
│   ├── media.md
│   └── delivery.md
```

8.3 脚本职责拆分

| 脚本 | 作用 |

|------|------|

| `init_project.py` | 创建唯一项目目录，初始化 `brief/state/feedback` |

| `workflow_helper.py` | 统一维护节点状态、Gate、反馈、补充信息 |

| `task_builder.py` | 统一生成 specialist task package，减少 PM prompt 负担 |

| `review_calibration.py` | 记录人工评分并生成 machine vs human 分差文件 |

| `wecom_doc_sync.py` | 将项目产物镜像到企业微信文档 |

| `wecom_gate_render.py` | 用固定模板渲染企业微信节点摘要 |

8.4 这一层的价值

脚本化编排带来的价值是：

- 降低 prompt 体积
- 减少 PM 手工维护状态的负担
- 提高状态流转和展示层的确定性
- 为评估、复盘和沉淀提供稳定文件结构

---

9. 评估体系

9.1 评估目标

评估体系围绕三件事展开：

- 工作流是否稳定
- 输出质量是否可接受
- 机器判断是否接近专业判断

9.2 核心指标

| 类别 | 指标 |

|------|------|

| 效率 | `token cost`、`time per agent`、`time to accept` |

| 流程 | `route accuracy`、`revision count`、Gate 通过率、项目完成率 |

| 质量 | Reviewer 通过率、Hard Gate 命中率、交付完整性 |

| 校准 | machine vs human `decision match rate`、总分差、单维度分差 |

9.3 评估方式

系统评估采用“固定案例 + 机器评审 + 人工复核 + 分差分析”的方式开展。

重点不是只看某次输出好不好，而是持续比较：

- 是否稳定回答 brief
- 高频错误类型是什么
- 优化后是否真实改善

---

10. AIPM 主导工作

从 AIPM 视角，本项目的核心不在于优化单个 prompt，而在于主导一整套 AI 协作系统的产品化设计与落地，主要包括：

10.1 流程设计

- 定义 `standard_workflow` 与 `single_specialist_workflow`
- 明确固定主链与单节点模式边界
- 确立 PM 只负责编排、不做专业内容的原则

10.2 角色拆分

- 定义 PM、Insight、Strategy、Reviewer、Creative、Media 的职责边界
- 避免多 Agent 职责交叉和内容污染

10.3 状态与文件体系设计

- 设计项目级目录结构
- 设计 `brief.md / state.json / feedback.md / delivery.md`
- 设计节点产物命名与落盘方式

10.4 Prompt 工程到脚本工程的迁移

- 将原本堆在 `AGENTS.md` 中的流程细节下沉到脚本
- 建立 `init_project / workflow_helper / task_builder / review_calibration`
- 把系统从“纯 prompt 驱动”升级为“prompt + script 混合编排”

10.5 质量机制建设

- 在 Strategy 后引入 Reviewer
- 设计 4 维评分 + Hard Gate
- 接入人工评分与分差分析
- 构建 Reviewer Calibration Loop

10.6 展示与协作入口设计

- 规划 dashboard 与企业微信双入口协作体验
- 设计文档镜像、节点摘要与外部展示规范
- 将展示层从自由生成收敛为固定模板，提升稳定性

10.7 端到端验证

- 验证标准主链
- 验证单节点链路
- 验证 Gate
- 验证返工机制
- 验证人工评分文件闭环

这部分工作体现的不是单点提示词优化，而是对 AI 多 Agent 系统的产品设计、流程设计、质量机制设计和可运营化能力。

---

11. 演进路线

11.1 工作流深化

- 持续优化 Reviewer 判断稳定性
- 优化节点摘要、展示与对外协作体验
- 提高返工机制和局部修改的稳定性

11.2 能力扩展

- 接入 `Content Agent`
- 接入 `Data Agent`
- 建立项目复盘与结果沉淀机制

11.3 知识与评估体系扩展

- 为每个 specialist 建独立文件知识库
- 将文件知识库升级为 RAG
- 建立更正式的 Agent 评估与优化体系
- 形成可复用的广告 Agent Team 产品化方法论

---

12. 一句话总结

本项目的核心价值，不在于“做了几个广告 agent”，而在于基于 OpenClaw 构建了一套可运行、可审核、可返工、可校准、可持续优化的广告多 Agent 协作系统。